CS-115: Programming 2
Coursework 1

Filippos Pantekis & Troy Astarte

Due: 09/03/2025 at 14:00

Welcome to the first programming assignment in Object Oriented Java! :-)

This coursework involves creating several classes and may be the largest project you've worked on so far! It really pays to approach tasks methodically and convince yourself that the code you have written behaves as expected, before moving on to following tasks. You should write a thorough series of test classes as you go (not just the one that is assessed!) and you can also use the output from Autograder to help guide the process. At times, you may get stuck. Don't panic! Programming languages, like spoken languages, take lots of practice to learn! :-)

This brief is divided into several tasks. These tasks are intended to break up a large piece of work into manageable chunks, to help you. It is strongly recommended that you complete each one before moving on to the next. The tasks generally **increase in difficulty**. There are **100** marks available for this coursework. **10** of those are reserved for code quality. Reffer to the rubric on Canvas.

For this assignment, you will have to make submissions to the department's Autograder system **refer to the assignment page for this coursework for further instructions.** As the grading is partly automated, any method, variable, class, and enum names provided in the following tasks, must be used exactly as shown (CaSe SeNsItIvE!). If no particular name is provided as part of the tasks, you are free to choose appropriate names.

Happy coding! :-)

What you may and may not use for this coursework

You should only need to use concepts you have been taught in the lectures in this coursework. You do not need to and **must not** use any of the following:

- **Inheritance:** No subclasses, extends, implements, etc. Only exception: you are allowed to override the toString() method.
- **External libraries:** Use of external libraries is prohibited.
- **Packages and Modules:** Do not put your classes within a Java package, and do not create module(s) for your code, as they may result in inaccurate Autograder outputs.
- **Content not in teaching materials:** Java offers a plethora of libraries/classes as part of the Standard Development Kit (SDK). You must not use any that we have not explicitly covered in the teaching materials of this module and CS-110. If you do use content we have not covered, you may get zero for the respective task and your work will come under suspicion of academic misconduct.

Scenario

In this coursework, you are developing software for a taxi operator to do basic management of their fleet, and support their guided city tour offerings for tourists.

Task 1: Creating a `Taxi` class [10 marks]

We will need to somehow represent a *Taxi* in the program. Create a class called `Taxi` to hold the following information:

- `taxiId`, an integer
- `numberPlate`, a String
- `mileage`, an integer
- `electricVehicle`, a boolean

Your `Taxi` class must have the above instance variables with the exact names and types provided.

The `taxiId` instance variable is an internal (to the object) ID, which starts from `499101` for the first `Taxi` object created by the program, and increases by `1` for every subsequent `Taxi` object. All other instance variables are required for a `Taxi` object to exist. In other words, it must be impossible to create a `Taxi` object without supplying values for each of the other instance variables.

The class must provide both an accessor and mutator for each of its instance variables, except the `taxiId` which is initialised automatically, and must only have an accessor.

The class must also provide a `toString()` method which should return a string that matches the below format **exactly**:

```
{taxiId; numberPlate; mileage; electricVehicle}
```

For example, for a `Taxi` where `taxiId=499101`, `numberPlate=AA00BBB`, `mileage=1234` and `electricVehicle=true`, the output must be:

```
{499101; AA00BBB; 1234; true}
```

Task 2: Testing the `Taxi` class [10 marks]

Write the class `TaxiUnitTest` which contains code to test the `Taxi` class. You should include the following methods which print output to the command line from within the method, named exactly as follows:

- `public static void testConstructorAndGetters()`
- `public static void testSetters()`
- `public static void testTaxiIds()`
- `public static void testToString()`

In each, you should test the expected vs. actual output for the corresponding aspects of the Taxi class, perhaps with a variety of values to check edge cases. For this task you are not allowed to use `JUnit`.

Your printed messages should be of the form:

```
<data_name>; Expected:  <exp_value>; Actual:  <act_value>; <outcome>
```

Here, the triangle-bracketed text is a placeholder and should be replaced with values, and the outcome text should either be the word "PASS" or "FAIL". For example:

```
numberPlate; Expected:  CE12GIK; Actual:  CE12GIK; PASS
```

You do not need to update this class as you continue to modify the `Taxi` class in later tasks, nor create tests for other classes you make. However, you are *strongly* encouraged to continue the unit testing approach throughout this coursework.

Turn over ☞

Task 3: Expanding the `Taxi` class [10 marks]

Every taxi has an assigned driver associated with it. Create a `Driver` class to hold the following information:

- `fullName`, a String
- `licenseCode`, a String

A `Driver` object should not be creatable without values to both attributes being supplied. Moreover, both should have an accessor and a mutator. Like with all classes you write for this assignment, you must ensure this class follows the principles of encapsulation.

Besides a driver, each taxi must also have a type. The company has three types of taxi: `SMALL`, `REGULAR` and `MINIBUS`. To represent this, you should create an enum called `TaxiType` with these three values.

In your `Taxi` class, create an attribute to hold the `Driver` of that taxi and another to hold the type of taxi it is. It should not be possible for an instance of the `Taxi` class to be created without a type supplied, however it should be possible for a `Taxi` to be created with, and without a `Driver` supplied. Think of the implications this requirement has for constructors.

Both of these instance variables should have an accessor, but only the driver instance variable should have a mutator. Do not modify `toString`.

Task 4: Adding some methods... [15 marks]

Time to start adding functionality to our classes.

The license identifier of a `Driver` is a string, which will always be exactly 10 characters long. The last 3 characters of that string, determine what type of taxi a driver can drive, as follows:

- If the 8th character from the left is 'M', the driver can drive a MINIBUS
- If the 9th character from the left is 'R', the driver can drive a REGULAR taxi
- If the 10th character from the left is 'S', the driver can drive a SMALL taxi

Note: A driver may be able to drive multiple different types of taxi.

Write a method in the `Driver` class called `canDrive` that takes a `TaxiType` parameter and returns true if the driver can drive it, or false otherwise.

Besides knowing each driver's ability to drive a taxi type, we want to have a way of calculating the cost of a ride. Create a method in your `Taxi` class called `calculateCost` that takes in an amount of miles, and returns the cost of driving the taxi this length. For some distance M the cost is calculated as follows:

- If the taxi is of type `SMALL`: cost $= M \times 0.93$
- If the taxi is of type `REGULAR`: cost $= M \times 1.21$
- If the taxi is of type `MINIBUS`: cost $= M \times 2.55$

A 30% discount is applied to the calculated cost, if the taxi is electric.

Think of how the `TaxiType` enum may support the implementation of this feature.

Turn over ☞

Task 5: Wait... Where is the data? [20 marks]

To answer the question: they're in a file!

This task involves reading a file. File reading will be covered in lectures after the release of this coursework. Prioritise other tasks until the content is covered.

For this task you have to write a file reader to read a text file and parse its contents into objects in your code. A sample input file is provided on Canvas, called `taxis.txt`. Download the file and place it in a local (to your project) location, where it can be read.

Create a new class called `FileDataLoader`, which contains a static method that takes a `File` parameter only, and returns a list of `Taxi` objects. In this method, write code to read the parameter file and create a list of taxi objects doing so.

Note: For this task you will also have to deal with exceptions. For the purposes of this task, you should have a try-catch block in this method and place all your file reading code in the try block. If an exception happens, you should catch it, print its stack trace, and **return null**.

The file you need to read for this task consists of multiple lines. Each line contains information about a taxi and its driver, and has the below **format**:

```
numberPlate,mileage,isElectricVehicle,taxiType,driversName,driversLicenseID
```

For example, consider the below line:

```
AB12CDE,131886,false,MINIBUS,John Doe,GwxqVUQMxS
```

This line should result in a Taxi object created which has the numberplate `AB12CDE`, mileage `131886`, is not an electric vehicle, is of type `MINIBUS`, and is driven by a Driver who's name is John Doe and who's license number is `GwxqVUQMxS`.

You may assume that the comma character only appears as a delimiter between fields in a file, and that all the data in the file is valid and well formed. You do not have to take into account files that do not meet this specification.

Note on Top Marks

You may have noticed (by examining the data in the sample file) that the same person (same name and license identifier) may be a driver for multiple different taxis. For this task, the top marks are available for solutions that make use of the minimum number of Driver objects needed. For instance, if the person "John Doe" is a driver for 5 different taxis, then only one Driver object needs to be created for John Doe, which can then be linked to all the taxis they are a driver for. Many ways exist to implement this and you are free to choose the one that is more natural to you.

Note on Testing

You do not need to submit testing code for this task, but is strongly recommended that you test the code thoroughly and ensure it works as expected. Your testing may involve creating input files of your own to test edge cases. File IO can be tricky, so allocate some extra time to check it works as expected!

Task 6: Tourist attractions!

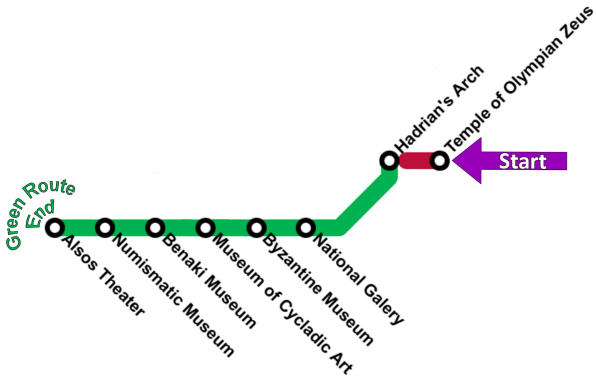
As part of their offerings, the taxi company offer a fixed city tour route (with branching sub-routes). When a passenger is picked up from one of the locations along these routes, they are given an offer to take a guided tour with the taxi for a fixed price.

The taxi operator wants the program to identify the most profitable route, given a starting location along the tour route. The company works out the price of the route by assuming the distance between each stop is exactly 1.5 miles, and then using the standard price calculation described earlier.

Turn over ☞

For example, let's say that the fixed tour route the company offers is as shown below:

Suppose that a passenger is picked up from the "Temple of Olympian Zeus". Starting from that point, the most profitable route is 7 stops long, as seen below, and would be costed the same as a $1.5 \times 7 = 10$ mile long taxi ride:



Note, routes are always directional, which means that from a given stop on the route we can only move one way. In the above example, we can only move towards the left. It's **not** possible to move from the "Temple of Olympian Zeus" towards the start point.

To help you dissect this task, it has been split into parts below.

Part 1: Creating a necessary class [5 marks]

For this task, you have to represent the fixed route map using objects, for your program to use later. The map should be represented in a manner similar to a linked list.

Create a class called `TouristAttraction`, which has the following attributes:

- The name of the attraction (a string)
- A list of `TouristAttraction`(s) that are next to this one (i.e., come immediately after this one on the map, in the direction of the map).

Additionally, your class should have the below methods:

- A method called `getName` that returns the name of the `TouristAttraction`.
- A method called `addNext` that takes a `TouristAttraction` parameter and adds it to the list of next attractions.
- A method called `getNext` that returns the list of `TouristAttractions`.

Your class should have no mutators. Moreover, the class must be constructible only by providing it the name of the attraction, and nothing else.

Turn over ☞

But wait, where is the data this time?

Note: The use of `CourseworkUtilities.java` described below is not assessed. This utility is only provided to help you with your development, testing and debugging and may come handy when you do Part 2 later on, which *is* assessed.

In the assignment page on Canvas, you will find the file `task_6_map.txt` and a Java file called `CourseworkUtilities.java`. Download both of these files and:

- Move the `CourseworkUtilities.java` file in the `src` folder of your project (alongside the other classes you have created).
- Move the `task_6_map.txt` file to a place where it can be read from.

In your code, you can call the method `CourseworkUtilities.loadTouristAttractions(file)` where `file` is a `File` object, pointing to the location of your `task_6_map.txt` file. This method reads the file and constructs the linked data structure, and then returns a `TouristAttraction` object, which is the first one in the line of the map (i.e., the start).

Bonus feature!

In the file `CourseworkUtilities.java` there's an additional method called `convertToDotGraph` that takes in a parameter `TouristAttraction` and returns a `String`. This method produces some text which, when pasted in a tool such as this: `https://dreampuf.github.io/GraphvizOnline/`, can be visualised nicely into a structure. While the visualisation varies and will not match the map structure shown in previous figures, it should still offer information about how the objects are linked together. If you find it hard to depict how `TouristAttraction` objects are chained together, you can call this method, give it a TouristAttraction object from which to start, print the returned `String`, and then copy the printed text into the visualisation tool.

Notes on the given code

The file `CourseworkUtilities.java` contains advanced Java code and may not follow the good code practice we've learned. You are not expected to understand how this code works, nor what it does. You should treat the code as a "black box" and rely only on using the relevant methods from it. You are not allowed to copy/use this code for other tasks in this coursework. You may only use (call) public methods from this class, and you must not change the code in this file in any way.

Part 2: Recursively processing the map [20 marks]

This part is challenging. Allow some extra time to think of the algorithm on paper before you start working on the code.

For this part, you have to write a method called `calculateMostProfitableRouteCost` that takes in a `TouristAttraction` and a `Taxi` parameter, and returns the cost of the most proffitable (longest) route, starting from the given `TouristAttraction`. The method must work out the length of the most profitable route using recursion, and then perform necessary calculations to work out how much this route costs for the given taxi. This method should be static and be part of its own class called `TaxiRouter`.

Hint

Although this part requires your code to use recursion, this does not mean that the method `calculateMostProfitableRouteCost` is the one that needs to be recursive. This method must have the signature mentioned above, but you are free to create other helper method(s). Likewise, you can have the recursive code in a method called by the `calculateMostProfitableRouteCost` method.

Turn over ☞

Submission Instructions

The deadline for this assignment is **09/03/2025 at 14:00**. You must both submit your code to Canvas, and also make *at least* one submission to Autograder before the deadline.

The above deadline is firm. No late submissions of the assignment are accepted without extenuating circumstances.

On Autograder

You should be using Autograder as you go through the tasks, to help guide your development. For the purposes of marking, you are expected to make **at least one** submission to Autograder before the deadline, containing **the final version of your code**. In that submission, you should submit any and all of the below files that you have worked on:

- `Taxi.java`
- `TaxiUnitTest.java`
- `Driver.java`
- `TaxiType.java`
- `FileDataLoader.java`
- `FileDataLoader.java`
- `TouristAttraction.java`
- `TaxiRouter.java`
- `CourseworkUtilities.java` *(although this code is given, you should have it in your project if you're using methods from it)*

Autograder is an integral part of marking your work. If you do not submit to Autograder, the submission will not be considered valid and will receive a mark of **0**. Instructions for Autograder are available through a link on the Coursework page. You can access Autograder here: `https://csautograder.swansea.ac.uk/`. Feedback for this assignment will be given during Lab 6 and by Autograder while using. You will receive your mark through a rubric on Canvas which will give further context. You may also discuss your work during drop-in hours.

On Canvas

Submit a **single .zip file** containing all the `.java` file(s) you created for your program in the root directory. **Only .zip files will be accepted!** Other formats may result in a mark of 0 for the assignment.

It is your responsibility to ensure the correctness and integrity of your submission. After making a submission, download it again, and double-check that it contains the final version of your work, as you intended. You can make multiple submissions before the deadline on Canvas. Only the last one (most recent), that was made before the deadline, will be considered.

Generative AI Usage Notice

⛔ The use of generative AI is **forbidden** for the entirety of this assignment.
For further information consult the university's Academic Integrity pages and if you require clarification please get in touch.

Academic Integrity Disclaimer

By submitting coursework, electronically and/or hardcopy, you state that you fully understand and are complying with the University's policy on Academic Integrity and Academic Misconduct. The policy can be found at `https://myuni.swansea.ac.uk/academic-life/academic-misconduct`

If you are unsure about Academic Misconduct make sure you have watched the induction lecture recording carefully. Handing in your coursework after the deadline will result in a mark of zero.